A3: LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

Progress report for meeting 2 (status: 23 October 2026)

1. Summary

Since meeting 1 we built every component of the plan we presented there, tested each one on real PostgreSQL servers, and prepared the full evaluation so that it can run unattended. In one sentence: **the fuzzer, its oracles, its triage, the LLM arm and the bug benchmark all exist and work; what remains is running the hour-long campaigns and writing their numbers into the paper.**

Plan item from meeting 1	Status
1. Grammar arm running for hours	done (6-hour hunts on master; 99.7 % valid statements)
2. Metamorphic oracles on derivation trees	**done**: TLP (WHERE, HAVING), NoREC, 10 expression rewrites, CTE toggle, table fence
3. Cross-version differential (master vs release)	**done**: every statement also runs on PostgreSQL 18.6
4. Triage pipeline	**done**: reproduce, reduce (statements, clauses, tokens), report, bisect, precision ledger
5. Commit-directed LLM loop with diff coverage	**done**: miner, cached prompts, validation gate on the server, repair rounds, cost ledger
6. Re-injected bug benchmark + statistics	**done**: 23 real fixes reverted, 20 observable, verified on 28 servers
7. Stretch: plan-shape feedback	**done** (plan shapes guide the corpus and are a metric)

Real bugs so far (both previously unknown, both with patches, filing in progress):

- **BUG-02, wrong result** (PostgreSQL 17, 18.6, master): the planner folds `'(,)'::int4range @> x` to `true` and `'empty'::int4range @> x` to `false` even when `x` is NULL; the correct answer is NULL. Found by our new tree-based expression rewrite. This is the paper's motivating example.
- **BUG-01, assertion failure** (17, 18, master): `JSON_TABLE` does not assign collations to its context item and `PASSING` arguments; a row comparison over such a value fails `Assert(list_length(rcexpr->inputcollids) == nopers)`. The new RQ3 smoke run **found it again on its own** within four minutes.

2. What we built (and why each piece matters)

2.1 Tree-based oracles (the main technical contribution)

The grammar engine now records the exact token span of every node of the derivation tree. Oracles therefore rewrite **one typed node** instead of re-parsing SQL text, which lets them work inside derived tables, CTEs, set-operation arms and `IN`/`EXISTS`/`LATERAL` subqueries, where text-based tools such as SQLancer cannot.

Oracle	Rewrite	Must hold
TLP on WHERE	`WHERE p` -> three copies with `p`, `NOT p`, `p IS NULL`, `UNION ALL`	same multiset
TLP on HAVING	same on `HAVING`; the other side keeps `HAVING TRUE`	same multiset
NoREC	`count(*) ... WHERE p` vs `sum(CASE WHEN p THEN 1 ELSE 0 END)`	same number
Expression rewrites (10)	`e -> COALESCE(e,e)`, `(SELECT e)`, `e+0`, `NOT NOT p`, ...	same result
CTE materialisation	toggle `MATERIALIZED`	same result
Table fence	`t -> (SELECT * FROM t OFFSET 0)`	same result
Plan variants (18)	e.g. hash join only, no hash aggregation, forced parallel, JIT	same multiset
Cross-version	same test case on PostgreSQL 18.6	same multiset

False-positive control: checks are attached only where the rewrite is sound (no `LIMIT` without `ORDER BY`, no volatile functions, order-sensitive aggregates excluded); a mismatch counts only if it repeats; digests ignore representation choices no plan controls (numeric scale, float `-0`).

2.2 Triage

`pgfuzz triage` reproduces each candidate three times, then reduces it with delta debugging on statements, hierarchical reduction over the parenthesis tree (clauses, list items, groups) and single tokens, keeping a step only if **the same** failure (same assertion, message, oracle and plan variant) occurs. For a metamorphic candidate the two queries are reduced together. Example: the BUG-02 candidate went from 4 statements / 107 tokens to 1 statement / 36 tokens in 307 runs. Every verdict goes into `triage/verdicts.yaml`, from which `pgfuzz precision` computes the precision of each oracle (RQ2).

2.3 The LLM arm (commit-directed grammar extension)

1. **Mine** recent master commits that change the planner/executor/parser and add regression tests; record their changed C lines (“diff-coverage targets”).
2. **Propose**: Claude reads one commit plus a cached system prompt (grammar format, builtins, conventions, schema, whole grammar; ~9,000 tokens) and answers in a JSON schema: rationale, grammar fragment, 1-5 seed queries.
3. **Gate** on the server: the fragment must parse, be reachable (used >= 5 times in 150 sampled test cases), be valid (>= 60 % of its statements run), and, on a gcov build, execute more of the commit’s changed lines than the grammar without it. Seeds must run.

4. **Repair**: a failing proposal goes back with the gate's findings (parse error, top error messages, uncovered lines), up to 3 rounds.
5. **Ledger**: every round (tokens, cost, full gate report) is one JSON line, which gives RQ4 directly (`pgfuzz llm report`).

Manual mode needs no API key (prompt written to a file, answer pasted back); stored responses are replayed for free, so every number is reproducible. Live test: the gate accepted a valid fragment (251/251 statements valid) and rejected an invalid one with the server's own error message.

2.4 RQ3 benchmark: real bugs put back into PostgreSQL

We took every fix in query processing between 31 Jul and 16 Oct 2026 that adds a regression test and reverts cleanly: **23 fixes**. We built three kinds of server from the same commit (`26bd08fb`, 16 Oct):

- **fixed** (the base), **injected** (all 23 reverted; the campaign runs here),
- **solo** servers, one per bug (only that fix reverted), to attribute findings.

For each bug we wrote a self-contained PoC from its regression test and ran it on all three kinds (`pgfuzz bench verify`):

Symptom on the solo server	Bugs
assertion failure	4
internal error (XX000)	5
wrong result	**8**
missing error (returns rows instead of an error)	2
wrong error	1
plan-only / not triggerable with built-in types (excluded)	3

20 of 23 bugs are observable, and only 9 of those 20 can be seen by a crash-only fuzzer, which is exactly the argument for oracles. On the injected server every PoC shows the same symptom as on its solo server (no masking). `pgfuzz bench attribute` assigns each campaign finding to a bug by replaying it on the fixed and the 23 solo servers.

3. Results so far (smoke runs; full campaigns pending)

- Smoke run 1 (4 min per arm, injected server): the grammar arm **rediscovered BUG-01**; the oracle arms produced 5 metamorphic and cross-version candidates. Triage showed they were **false positives of our own oracles**:
    a. TLP on `HAVING` dropped `HAVING` from a query without `GROUP BY`, which changes it from a one-group aggregate to a plain query; (b) `DISTINCT` over `numeric` keeps `21.0000` or `21.00` depending on the plan. Both fixed, with regression tests, and recorded in the precision ledger. This is RQ2 data and a lesson for the paper: *a richer grammar finds the corners where two "equivalent" queries are not equivalent; oracle precision is the bottleneck*.
- Smoke run 2 (15 min per arm, fixed oracles): see Section 3.1.

- Reach after 270 s: each grammar arm 101 SQL features and > 2,000 plan shapes; the random arm 17 and 17. Validity of the grammar arm: 99.7 % (19,780 of 19,846 statements).

3.1 Smoke run 2

15 minutes per arm on the injected server, all oracle fixes applied, then `pgfuzz bench attribute`:

Arm	Test cases	Valid statements	SQL features	Plan shapes	Benchmark bugs found
Random/Mutation	174,104 (one statement each)	26.3 % (77.7 % of those that parse)	17	22	0
Grammar	16,841	99.6 % of 56,233	101	5,416	0
Grammar+Oracles	15,861	99.7 % of 42,852	101	5,968	**1** (rb-dfcf6fa7 at 158 s)
Grammar+Oracles+LLM (no fragments yet)	15,800	99.7 % of 42,684	101	5,937	**1** (rb-dfcf6fa7 at 160 s)

- `rb-dfcf6fa7` ("could not find pathkey item to sort") has the same message as `rb-8b29b8da`; replay on the two solo servers assigned it correctly.
- Two random-arm "finds" were **our own fault**: a PoC replay we ran during the campaign crashed the shared server, and the harness blamed the random arm's current statement. Fixed: the executor now reads the PID of every `TRAP` / signal line and ignores crashes of other backends (a parallel worker of our own query still counts).
- A cross-version mismatch, `CORR(1.0/3, 1.0/3)` = NULL on master vs 1 on 18.6, is an intended change in master; those functions are now excluded from the cross-version comparison.
- One internal error ("Append child's targetlist doesn't match Append") did not reproduce anywhere (flaky); it stays in the ledger.

Precision ledger so far (`pgfuzz precision`): 18 candidates, 3 real, 13 false positives or intended changes from 6 classes that are now fixed, 2 flaky. Precision 0.19 before the fixes, 1.00 on the candidates whose class remains.

4. Engineering decisions worth knowing for the viva

- **One commit for everything.** The coverage build, the benchmark base and the end of the commit-mining window are all `26bd08fb`, so RQ1-RQ4 measure the same code and diff-coverage line numbers line up.
- **No leakage into the LLM.** `llm mine --exclude bench/reinjected/commits.txt`: the model never sees the fixes of the bugs RQ3 re-injects.
- **No mixed coverage.** gcov writes counters next to the build; with parallel trials they would merge. Each coverage server gets its own `GCOV_PREFIX`.
- **No shared reference tables.** Parallel trials share one 18.6 server, so each worker gets its own database there (created on demand).
- **Same oracles for every arm.** Arms differ only in the generator, so RQ1-RQ3 compare generators, not oracle sets.

5. Code and tests

- ~9,200 lines of Python in `src/pgfuzz`, 122 unit tests (all pass), grammar of 78 rules / 419 alternatives / 13 types.
- New since meeting 1: `grammar/rewrite.py` (oracles), `reference/xversion_executor.py`, `triage/` (reduction, reports, precision), `llm/` (miner, prompts, client, gate, diff coverage, loop), `bench/` (benchmark), `configs/rq1.yaml`, `configs/rq3.yaml`, `scripts/build_reinjected.sh`, `rq1_servers.sh`, `rq3_servers.sh`, `ref_server.sh`.
- `docs/EXECUTION_PLAN.md`: the exact commands for every campaign, expected times, what to screenshot and what to send back.

6. Paper

The FSE-format draft now has all text that does not depend on final numbers: title, abstract, introduction with the real motivating example (BUG-02), background, problem statement, approach with the workflow figure, evaluation design (RQs, benchmark, arms, measurements, statistics), discussion, threats to validity, related work (13 references, each checked against the publisher or author pages), conclusion and data/AI statements. Numbers still to come are marked in blue (`\num{}`). It compiles with `make` in `paper/` (ACM `acmart`, review mode) to 12 pages with the references starting on page 11, so the result tables have room; `make check` reports no LaTeX errors, no undefined references and no banned words. The current PDF is `docs/PAPER_DRAFT.pdf`.

7. Plan to meeting 3 and the deadline

When	What	Who
24-26 Oct	Phase 0-1 of the execution plan: builds, LLM extension (~$5-10 API)	team runs, Claude analyses
26-27 Oct	Phase 2: RQ3 campaign overnight (4 arms x 5 trials x 1 h)	team runs
27-28 Oct	Phase 3: RQ1/RQ2/RQ4 campaign overnight (6 arms x 5 trials x 1 h)	team runs
28-30 Oct	Triage every candidate, file BUG-01 and BUG-02 upstream	together
meeting 3 (2-6 Nov)	all RQ tables and figures, paper with numbers	
12 Nov	presentation	
13 Nov	paper and repository	

Risks: (1) campaigns produce new oracle false positives: triage handles them and they count as RQ2 data; (2) the LLM API is unavailable: manual mode works without a key; (3) upstream does not respond before 13 Nov: we report the filing, the reproducer and the patch.